



Testeo Automatizado

TESTEO AUTOMATIZADO

Unit Testing



AGENDA

- Pruebas Unitarias
- TDD – Test Driven Development
- Presentación TP N^º1



“Los malos programadores tienen todas las respuestas. Los buenos probadores tienen todas las preguntas”

Gil Zilberfeld

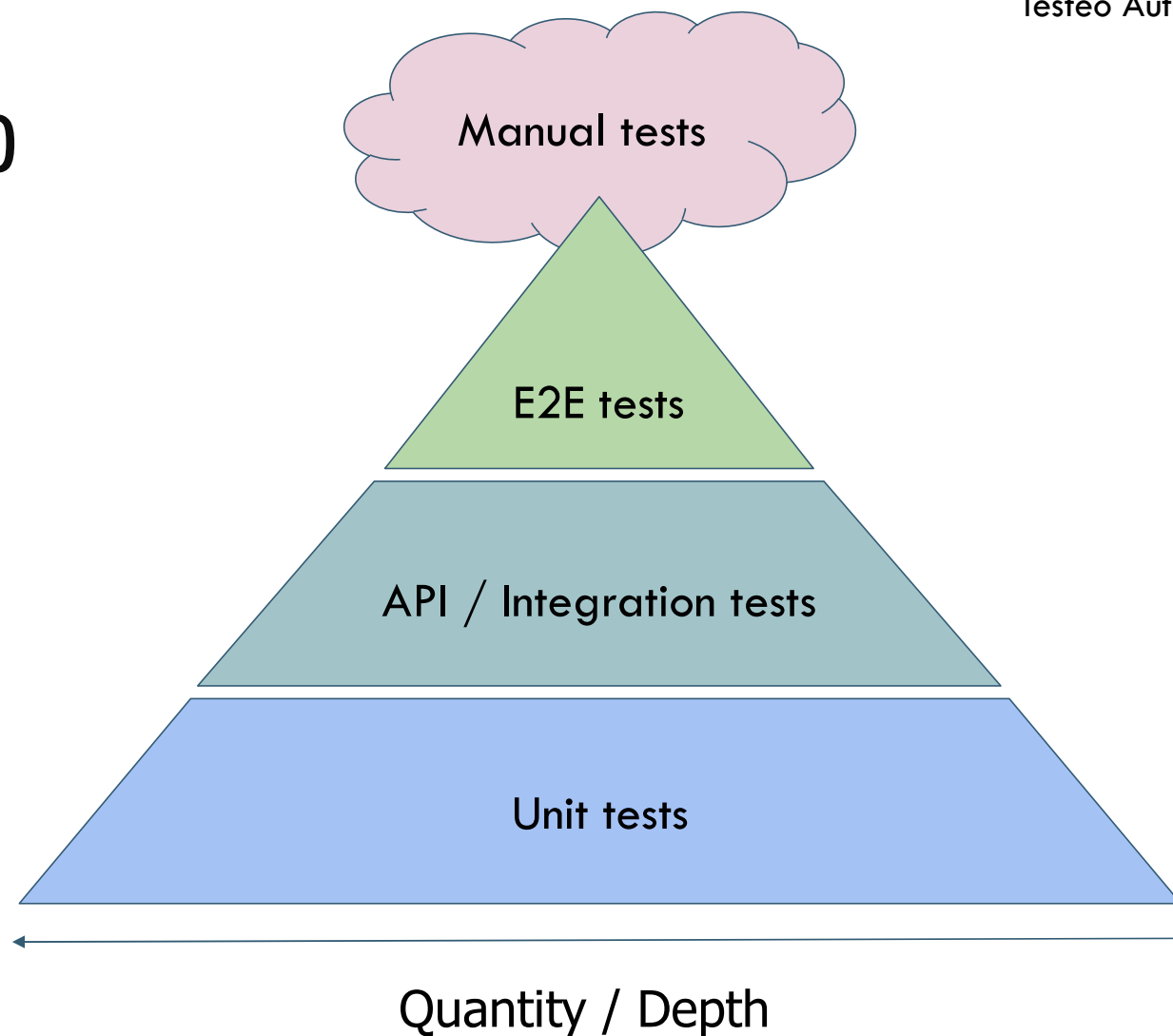




TESTING PYRAMID

Slower execution
Coverage +
More difficult maintenance
More brittle
Longer to write

Faster execution
Coverage –
Easier maintenance
More robust
Quicker to write





QUE ES EL TESTING UNITARIO?

- Testear la menor Unidad de código:
 - ↪ Una función
 - ↪ Una clase (en OOP)
 - ↪ Un modulo
- El Tamaño de Unidad depende de nuestra decisión
- Generalmente realizada por los **developers**
- Debería ser
 - ↪ rápido
 - ↪ confiable / robusto
 - ↪ independiente de otro recurso
 - ↪ Involucrado con el código bajo prueba
 - ↪ escrito en el mismo lenguaje que el código bajo prueba



COBERTURA DE TEST UNITARIO

- Cantidad de código que fue testeado por las pruebas unitarias
- Expresada en
 - ↪ % de líneas de código
 - ↪ % de ramas de código
- Generalmente integrada con CI/CD
 - ↪ define umbrales
 - ↪ nivel mínimo obligatorio de cobertura



PATRON DE TESTING AAA

- A de **Arrange**
 - ↪ Setear los datos iniciales
 - ↪ Setear el escenario
- A de **Act**
 - ↪ Ejecutar el comportamiento a probar
- A de **Assert**
 - ↪ Validar los resultados



TESTING CYCLE

- Setup with `before()` and `beforeEach()` hooks
 - ↪ `before()` runs once at beginning of scope (global, or suite level)
 - ↪ `beforeEach()` runs once before each test case in scope
- Group tests in test suites with `describe()`
- Test with `it()`
- Teardown with `after()` and `afterEach()` hooks
 - ↪ `after()` runs once at end of scope (global, or suite level)
 - ↪ `afterEach()` runs once after each test case in scope



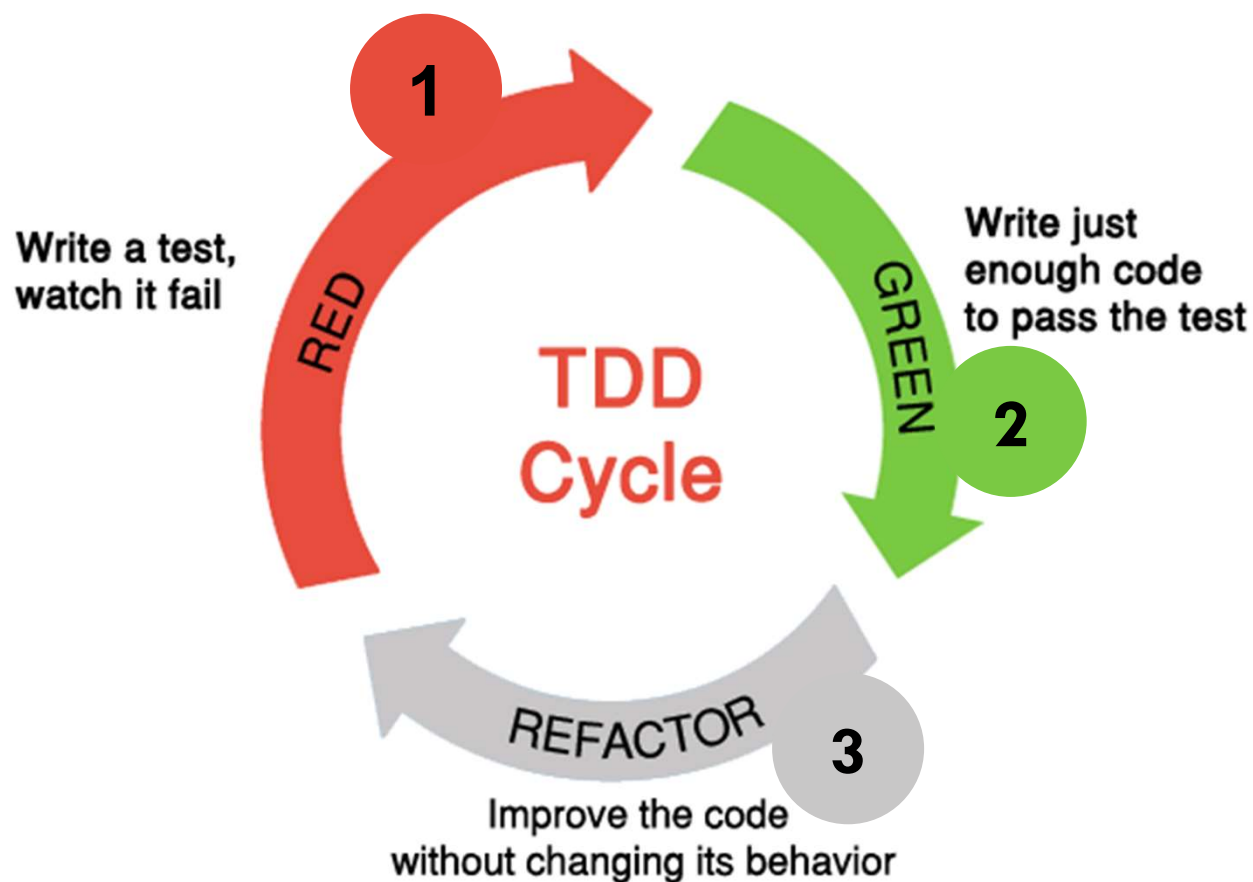
SETUP CODE COVERAGE FOR YOUR BACK END

- Investigate how to setup **nyc** tool for code coverage with Mocha JS
- Create a few examples of functions and unit tests
- Compute coverage and generate report





TDD - THINKING INSIDE OUT

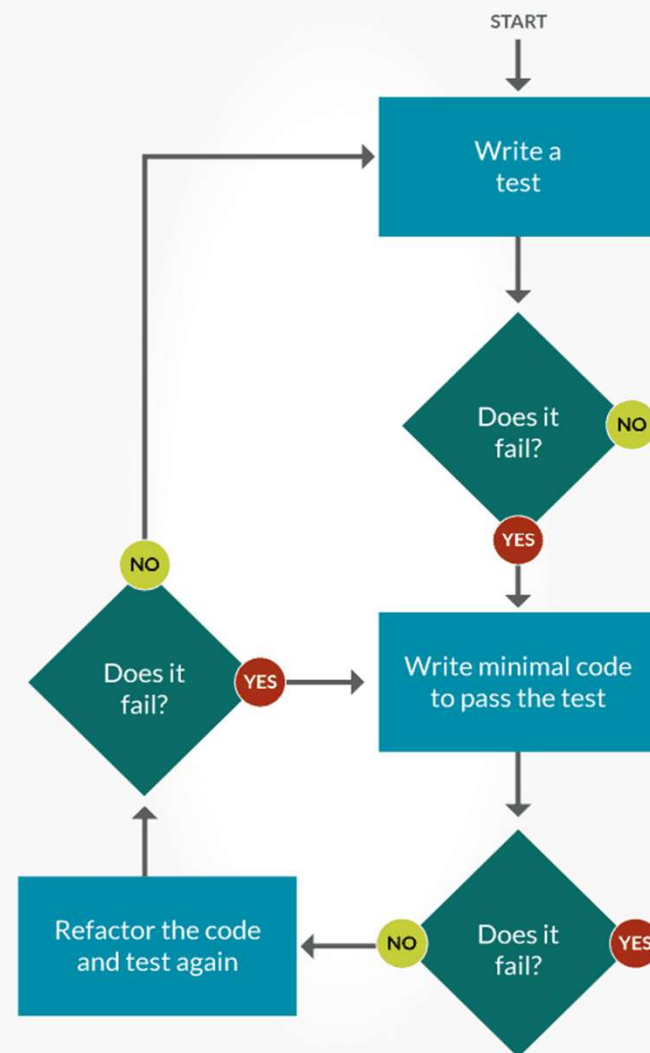




TDD - LIFE CYCLE

1. Write the test
2. Run the test (there is no implementation code, test does not pass)
3. Write just enough implementation code to make the test pass
4. Run all tests (tests pass)
5. Refactor
6. Repeat

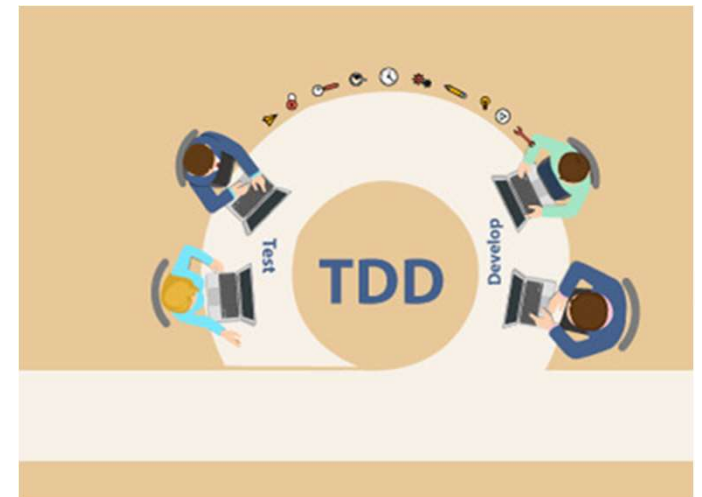
Testeo Automatizado





CONSIDERATIONS

- Test-driven development is not about testing.
- Test-driven development is about development (and design), specifically improving the quality and design of code.
- The resulting unit tests are just an extremely useful by-product.

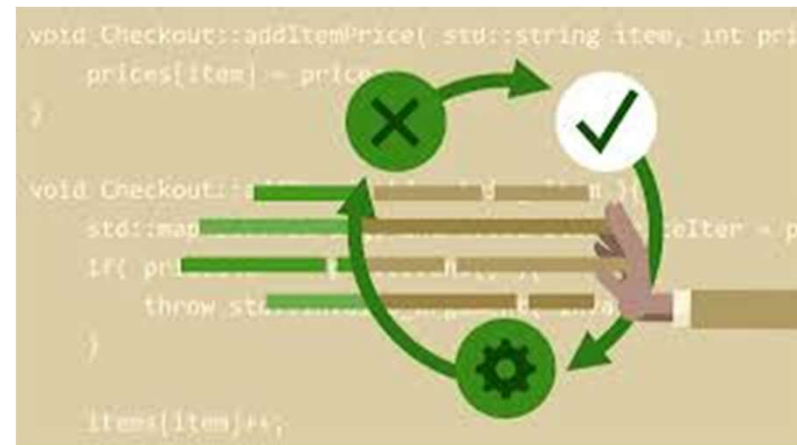




TDD - BEST PRACTICES

Practices have been separated into following categories:

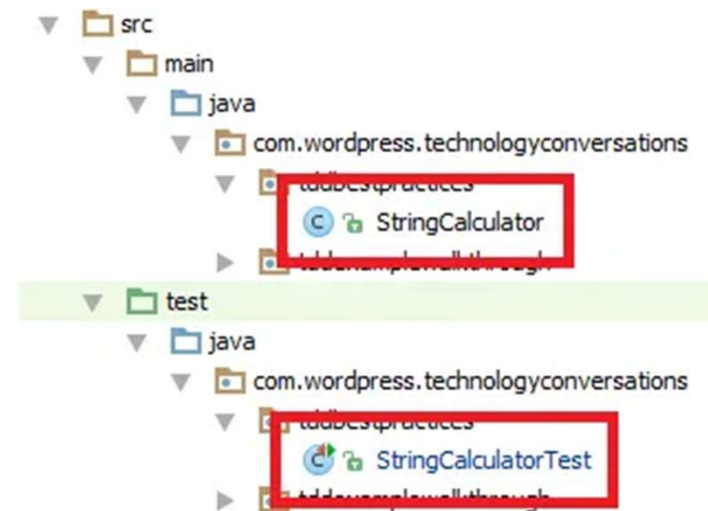
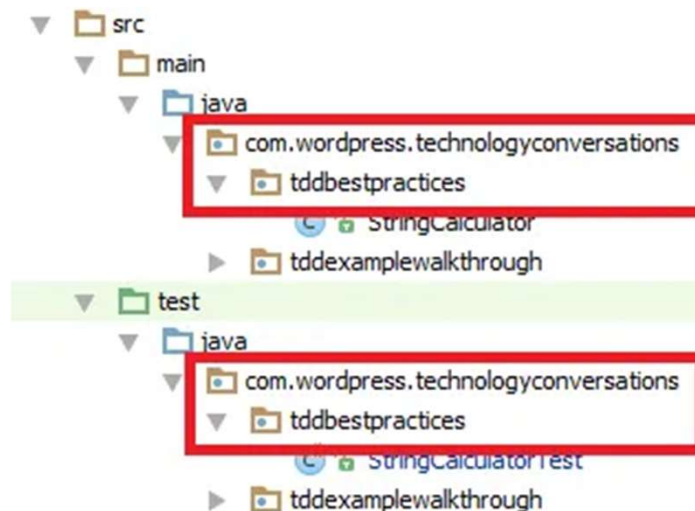
1. Naming Conventions
2. Processes
3. Development practices
4. Tools





NAMING CONVENTIONS

- Separate the implementation from the test code.
- Place test classes in the same package as implementation.
- Name test classes in a similar fashion as classes they test.
- Use descriptive names for test methods.





PROCESSES

- Write the test **before** writing the implementation code.
- **Only** write new code when test is failing.
- Rerun **all** tests every time implementation code changes.
- All tests should pass **before** new test is written.
- Refactor **only** after all tests are passing.





DEVELOPMENT PRACTICES

- Write the simplest code to pass the test.
- Write assertions first, act later.
- Minimize assertions in each test.
- Do not introduce dependencies between tests.
- Tests should run fast.
- Use mocks.
- Use setup and tear-down methods.
- Do not use base classes.



TOOLS

- Code coverage.
- Continuous integration (CI).
- Use TDD together with BDD.

JACOCO
Java Code Coverage



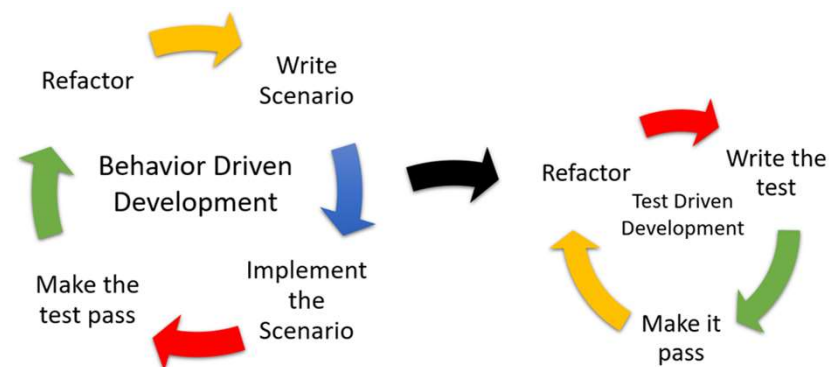
Jenkins

 **Clover**

 **Bamboo**

Cobertura

Hudson





Testeo Automatizado

MUCHAS GRACIAS